

ZESPÓŁ SZKÓŁ ELEKTRYCZNO- MECHANICZNYCH

Pracownia testowania i dokumentowania aplikacji

Testowanie aplikacji

Autor: Adrian Leśnik

Klasa: 4P

Prowadzący: mgr inż. Kamil Wojnarowski

NOWY SĄCZ 2026r.

Spis treści

Wstęp	4
1. Testy jednostkowe	5
1.1 Implementacja programu	5
1.2 Implementacja testów	6
1.3 Git	7
Podsumowanie	8

Wstęp

Testowanie oprogramowania polega na sprawdzaniu, czy program działa zgodnie z wymaganiami i oczekiwaniami użytkownika. Jego głównym celem jest wykrycie błędów, zanim trafią one do użytkownika końcowego.

Wyróżniamy cztery podstawowe poziomy testowania: testy jednostkowe (pojedyncze metody i klasy), integracyjne (współpraca modułów), systemowe (cały system) i akceptacyjne (zgodność z wymaganiami klienta). Testy dzielimy też na czarnoskrzynkowe, wykonywane bez znajomości kodu, oraz białoskrzynkowe, które wykorzystują wiedzę o jego budowie.

Do najważniejszych zasad testowania należą: testy pokazują obecność błędów, ale nie dowodzą ich braku, nie da się przetestować wszystkiego, a im wcześniej zaczniemy testować, tym taniej naprawimy błędy. Proces testowy obejmuje planowanie, projektowanie, wykonanie i ocenę wyników testów.

W pracy opisano testy jednostkowe na przykładzie programu napisanego w języku C# i testowanego w bibliotece xUnit.

1. Testy jednostkowe

Testy jednostkowe sprawdzają najmniejsze części programu, czyli pojedyncze metody lub klasy, w oderwaniu od reszty aplikacji. Wykonuje się je w trakcie pisania kodu, zwykle zaraz po napisaniu metody, oraz po każdej jego zmianie, aby upewnić się, że nic się nie zepsuło.

Przykład. Dla metody obliczającej cenę po rabacie test jednostkowy polega na wywołaniu jej z konkretnymi danymi i sprawdzeniu wyniku: dla ceny 100 zł i rabatu 20% oczekujemy 80 zł.

xUnit to darmowa biblioteka do testów jednostkowych na platformie .NET. Test to metoda oznaczona atrybutem [Fact], a wynik sprawdza się metodami klasy Assert, np. Assert.Equal lub Assert.Throws. Testy uruchamia się poleceniem dotnet test albo w Test Explorerze w Visual Studio.

1.1 Implementacja programu

Testowany program to klasa Sortowanie z metodą Sort, która przyjmuje tablicę liczb całkowitych i zwraca nową, posortowaną rosnąco tablicę. Użyto sortowania bąbelkowego.

```
using System;

namespace Sortowanie_Przez_Wstawianie
{
    public class Sortowanie
    {
        public int[] Sort(int[] array)
        {
            if (array == null)
            {
                throw new ArgumentNullException(nameof(array));
            }

            int[] result = (int[])array.Clone();
            int n = result.Length;

            for (int i = 0; i < n - 1; i++)
            {
                for (int j = 0; j < n - i - 1; j++)
                {
                    if (result[j] > result[j + 1])
                    {
                        int temp = result[j];
                        result[j] = result[j + 1];
                        result[j + 1] = temp;
                    }
                }
            }

            return result;
        }
    }
}
```

Metoda najpierw sprawdza, czy tablica nie jest null – jeśli jest, zgłasza wyjątek `ArgumentNullException`. Następnie tworzy kopię tablicy, żeby nie zmieniać danych wejściowych. Dwie zagnieżdżone pętle porównują sąsiednie elementy i zamieniają je miejscami, gdy lewy jest większy od prawego. Po każdym przebiegu największy element trafia na koniec, dlatego pętla wewnętrzna jest z każdym przebiegiem krótsza. Na końcu metoda zwraca posortowaną kopię.

1.2 Implementacja testów

Testy napisano w xUnit. Klasa `SortowanieTests` zawiera cztery testy sprawdzające różne przypadki.

```
using Xunit;
using System;
using Sortowanie_Przez_Wstawianie;

namespace Sortowanie_Przez_Wstawianie_Testy
{
    public class SortowanieTests
    {
        private readonly Sortowanie _sorter = new Sortowanie();

        [Fact]
        public void Test1()
        {
            int[] input = { 4, 2, 1, 3 };
            int[] expected = { 1, 2, 3, 4 };
            Assert.Equal(expected, _sorter.Sort(input));
        }

        [Fact]
        public void Test2()
        {
            int[] input = { 1, 2, 3 };
            int[] expected = { 1, 2, 3 };
            Assert.Equal(expected, _sorter.Sort(input));
        }

        [Fact]
        public void Test3()
        {
            int[] input = Array.Empty<int>();
            int[] expected = Array.Empty<int>();
            Assert.Equal(expected, _sorter.Sort(input));
        }

        [Fact]
        public void Test4()
        {
            Assert.Throws<ArgumentNullException>(() => _sorter.Sort(null));
        }
    }
}
```

Test1 sprawdza sortowanie zwykłej tablicy, Test2 – tablicy już posortowanej, Test3 – tablicy pustej, a Test4 – czy dla wartości null zostanie zgłoszony wyjątek `ArgumentNullException`. Po uruchomieniu `dotnet test` wszystkie cztery testy zakończyły się wynikiem pozytywnym.

1.3 Git

Do zapisywania zmian użyto systemu Git, a repozytorium znajduje się na GitHubie. Każdy test zapisano w osobnym commicie:

- 9b2fbf8 – sprawdzenie sortowania zwykłej tablicy
- b93d938 – sprawdzenie tablicy już posortowanej
- 0ae200d – sprawdzenie pustej tablicy
- 3f986c5 – sprawdzenie rzucania wyjątku dla null

Podsumowanie

W pracy opisano testy jednostkowe i pokazano je na przykładzie metody sortującej tablicę w języku C#. Napisano cztery testy w xUnit: dla zwykłej tablicy, tablicy posortowanej, pustej oraz dla wartości null. Wszystkie zakończyły się powodzeniem, a przebieg prac zapisano w czterech commitach w Git.